Prathamesh Brahma

CS23179

Lab 8 Extra Datasets

Aim:To perform Linear Discriminant Analysis (LDA) and Principal Component Analysis (PCA) separately on a dataset, apply Logistic Regression on the transformed features, and compare the performance of LDA-based and PCA-based models.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import roc_curve, roc_auc_score
```

```
path = "/content/drive/MyDrive/ML datasets/Breast_Cancer_dataset.csv"
df = pd.read_csv(path)
df.head()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	842302	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.1471
1	842517	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.0701
2	84300903	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.1279
3	84348301	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.1052
4	84358402	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.1043

5 rows × 32 columns

```
print(df.columns)
print(df.shape)
```

```
Index(['id', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
       'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'diagnosis'],
      dtype='object')
(569, 32)
```

```
df.columns = df.columns.str.strip()
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(" ", "_")

print(df.columns)
```

```
Index(['id', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
       'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave_points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave_points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave_points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'diagnosis'],
```

```
      dtype='object')
```

```
cat_cols = df.select_dtypes(include=['object']).columns
num_cols = df.select_dtypes(exclude=['object']).columns

print("Categorical:", cat_cols)
print("Numerical:", num_cols)
```

```
Categorical: Index(['diagnosis'], dtype='object')
Numerical: Index(['id', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
       'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave_points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave_points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave_points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')
```

```
df_encoded = pd.get_dummies(
    df,
    columns=cat_cols,
    drop_first=True
)

print(df_encoded.head())
```

```
        id  radius_mean  texture_mean  perimeter_mean  area_mean  \
0   842302        17.99         10.38          122.80     1001.0
1   842517        20.57         17.77          132.90     1326.0
2  84300903        19.69         21.25          130.00     1203.0
3  84348301        11.42         20.38           77.58      386.1
4  84358402        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave_points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  ...  texture_worst  perimeter_worst  area_worst  \
0         0.2419  ...          17.33           184.60      2019.0
1         0.1812  ...          23.41           158.80      1956.0
2         0.2069  ...          25.53           152.50      1709.0
3         0.2597  ...          26.50            98.87       567.7
4         0.1809  ...          16.67           152.20      1575.0

   smoothness_worst  compactness_worst  concavity_worst  concave_points_worst  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   symmetry_worst  fractal_dimension_worst  diagnosis_M
0          0.4601                  0.11890         True
1          0.2750                  0.08902         True
2          0.3613                  0.08758         True
3          0.6638                  0.17300         True
4          0.2364                  0.07678         True

[5 rows x 32 columns]
```

```
X = df_encoded.drop('diagnosis_M', axis=1)
y = df_encoded['diagnosis_M']
```

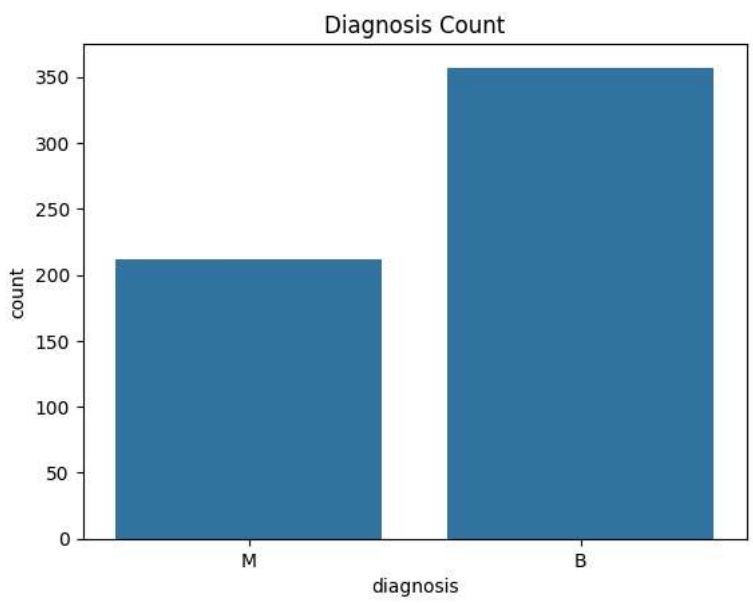
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

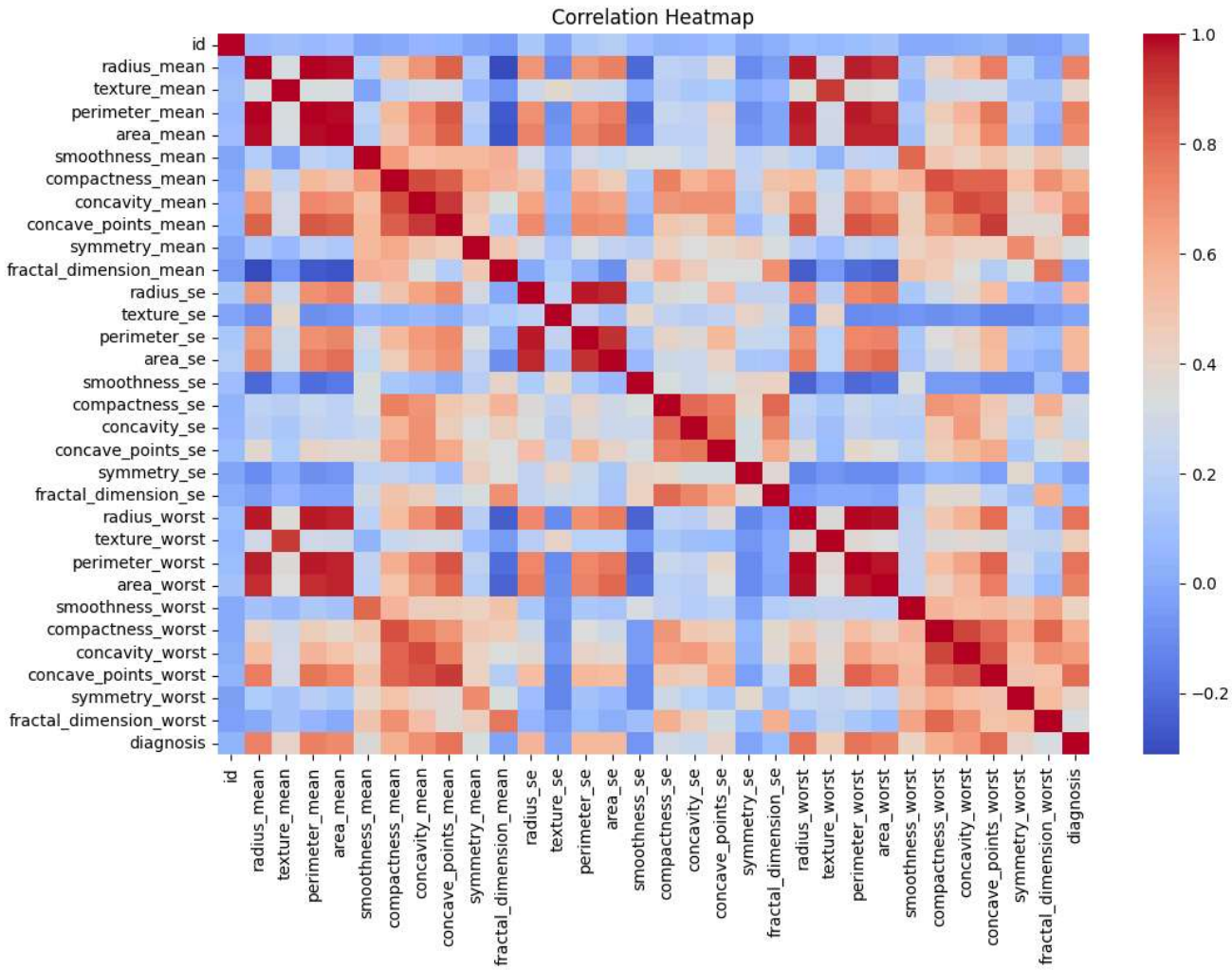
```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
model = SVC(kernel='rbf')
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.9824561403508771
Confusion Matrix:
 [[71  0]
 [ 2 41]]
```

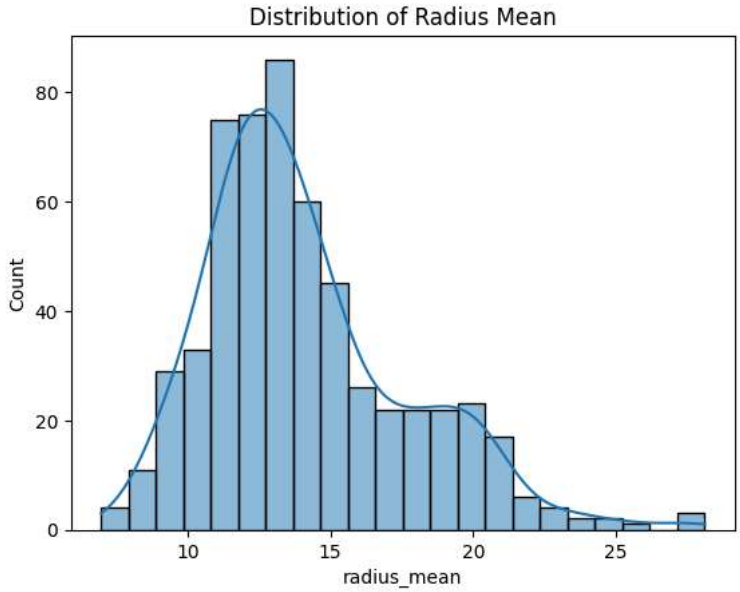
```
sns.countplot(x='diagnosis', data=df)
plt.title("Diagnosis Count")
plt.show()
```



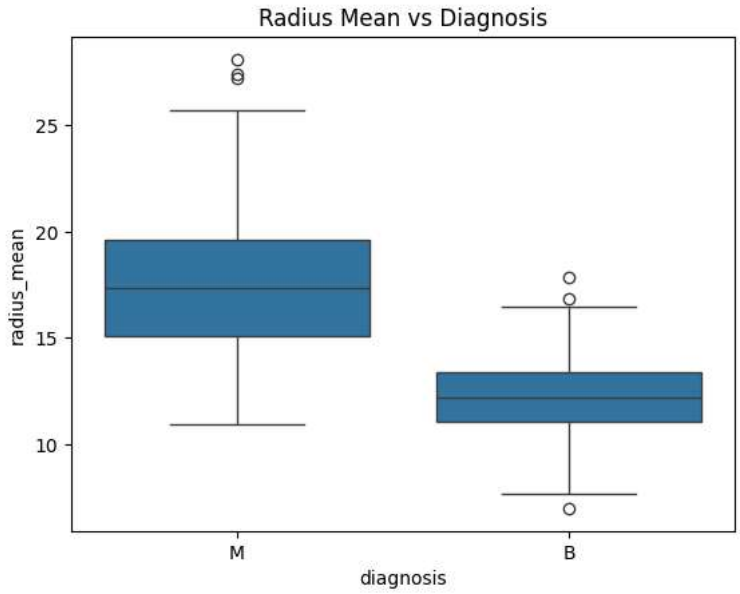

```
df_numeric = df.copy()
df_numeric['diagnosis'] = df_numeric['diagnosis'].map({'M': 1, 'B': 0})
plt.figure(figsize=(12,8))
sns.heatmap(df_numeric.corr(numeric_only=True), cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```




```
sns.histplot(df['radius_mean'], kde=True)
plt.title("Distribution of Radius Mean")
plt.show()
```



```
sns.boxplot(x='diagnosis', y='radius_mean', data=df)
plt.title("Radius Mean vs Diagnosis")
plt.show()
```




```
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})
```

```
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
svc_model = SVC(kernel='rbf', probability=True)
svc_model.fit(X_train, y_train)
```

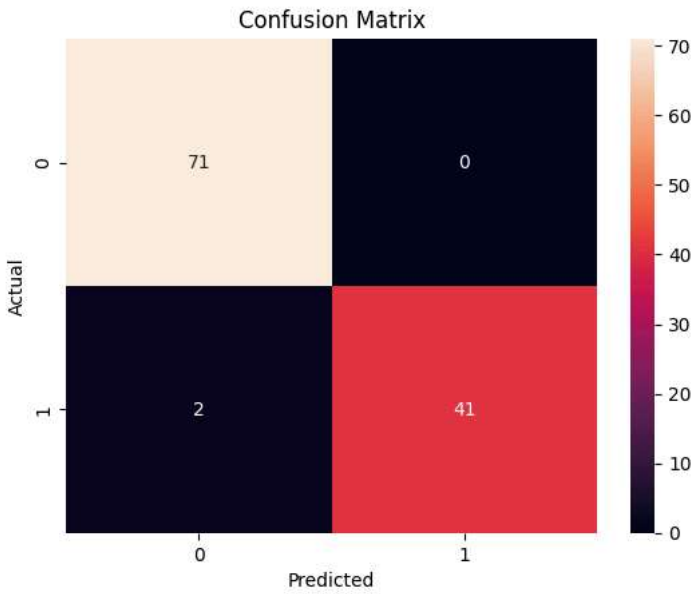
```
SVC
SVC(probability=True)
```

```
y_pred = svc_model.predict(X_test)
```

```
print("SVC Accuracy:", accuracy_score(y_test, y_pred))
```

```
SVC Accuracy: 0.9824561403508771
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        71
           1       1.00      0.95      0.98        43

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
model = SVC()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```